Dynamic Breadth First Search with Predictions

A Project Report

Submitted by

Shubham Kumar Verma **(Enrolment No: 22114092)**
Utkarsh Lohiya **(Enrolment No: 22114103)**

for the fulfillment

of

CSN-400B: B.Tech. Project

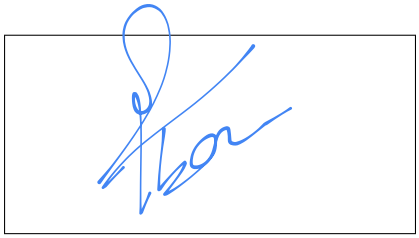
Under the guidance of

Prof. Shahbaz Khan
Assistant Professor,
Dept. of Computer Science and Technology

{shahbaz.khan,shubham_kv,utkarsh_l}@cs.iitr.ac.in

May 18, 2026

Department of Computer Science and Technology
Indian Institute of Technology Roorkee
Roorkee-247667



Prof. Shahbaz Khan

Acknowledgement

We express our sincere gratitude to **Professor Shahbaz Khan**, Assistant Professor, Department of Computer Science and Technology, Indian Institute of Technology Roorkee, for his invaluable guidance, constant encouragement, and generous support throughout this B.Tech. Project (CSN-400B). His deep expertise in the design and analysis of algorithms and his patient mentorship have been instrumental in shaping this work. Every discussion with him helped us understand the problem more deeply and pushed the research forward in meaningful ways.

We thank the Department of Computer Science and Technology, IIT Roorkee, for providing us with the computational resources, library access, and the academic environment necessary to carry out this research. We are also grateful to our fellow students and colleagues who offered feedback and constructive discussions at various stages of this project.

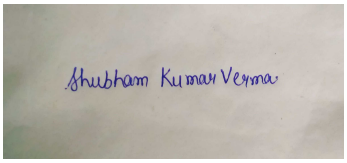
This work is done under the guidance of Professor Shahbaz Khan for the course titled CSN-400B (B.Tech. Project) at Indian Institute of Technology, Roorkee.

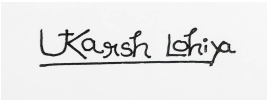
Declaration

We hereby declare that the project report entitled **"Dynamic Breadth First Search with Predictions"**, submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology** in **Computer Science and Engineering** at **Indian Institute of Technology Roorkee**, is an account of our original work carried out under the supervision of **Prof. Shahbaz Khan**, Department of Computer Science and Technology, IIT Roorkee.

We further declare that:

- The content of this report is based on our own research, carried out during the academic year 2025–2026.
- This report has not been submitted, either in whole or in part, to any other institution or university for the award of any degree, diploma, or other academic qualification.
- All sources consulted and cited in this report have been properly acknowledged.
- To the best of our knowledge, this work does not infringe upon any copyright.





Shubham Kumar Verma **Utkarsh Lohiya**

Enrolment No: 22114092 Enrolment No: 22114103

Department of Computer Science and Technology
Indian Institute of Technology Roorkee
May 18, 2026

Abstract

Given a graph $G = (V, E)$ having n vertices and m edges, we investigate the problem of maintaining its Breadth-First Search (BFS) tree from a fixed source $s \in V$, subject to an online sequence of edge insertions and/or deletions in the prediction model. Our approach leverages a *predicted* update sequence that can be preprocessed to build data structures based on anticipated updates, aiding online processing. We present algorithms for incremental (insertions-only), decremental (deletions-only), and fully dynamic (insertions and deletions) settings that maintain a BFS tree (parent and level information). Classically, without using predictions, the incremental and decremental BFS tree can be maintained in total $O(mn)$ time using ES Trees [9], resulting in amortized $O(n)$ update time. However, in the worst case, it requires $\Omega(m)$ time. Moreover, for maintaining BFS trees under fully dynamic edge updates, no algorithm better than trivial recomputation after every update is known, requiring $O(m)$ time per update. Further, assuming the combinatorial BMM conjecture, no algorithm can maintain incremental or decremental BFS polynomially faster than $O(mn)$ [1], even when the entire update sequence is known in advance [13].

Our complexity bounds are expressed in terms of error measures of the prediction, where *error vertices* are those having incorrectly predicted distance, with the corresponding difference called their *error*. We define the vertex prediction error η_v as the sum of the degree of *error vertices*, and the weighted vertex prediction error η_v^* as the *error weighted* sum of the degree of *error vertices*. Additionally, we define η_e as the number of incorrectly predicted edge updates.

For incremental and decremental BFS our algorithm requires respectively $O(\eta_v + \eta_e)$ and $O(\min\{m, \eta_v^* + \eta_e\})$ worst case update times using $O(mn)$ preprocessing time and space, and total update time of $O(\eta_v^* + \eta_e)$. We also present the space-efficient extensions of the above algorithms requiring $O(n^2)$ and $O(\frac{mn}{k} + n^2)$ for any $k \in [1, n]$, at the expense of additional $O(n \log n)$ and $O(nk)$ time in the worst case and total time, respectively. For fully-dynamic updates, our algorithm requires an update time of $O(\min\{m, \eta_v^* + \eta_e\})$ in the worst case. At its core, we extend the classical ES Trees [9] for batch updates and fully dynamic updates, which may be of independent interest. Notably, this simple extension of ES Trees for batch updates is sufficient to give a competitive prediction algorithm, which can thus be generalized to other graph problems. Additionally, we consider space optimizations and error correction mechanisms, which further improve the performance of our results.

Contents

1 Introduction

The area of dynamic graph algorithms deals with such graphs that evolve over time by a sequence of online updates, i.e., insertion or deletion of edges. The goal is to maintain the data structure or solution after every such update, much faster than trivially recomputing it from scratch after every update. An algorithm is called *fully dynamic* if it allows both insertion and deletion updates. It is called *partially dynamic*, i.e., *incremental* or *decremental* when the updates are limited to insertions or deletions, respectively. Some of the prominent graph problems studied in this area include connectivity [10, 15, 16, 18, 5], reachability [17, 9, 12, 3, 7], shortest paths [6, 29, 28], etc. Several such algorithms are also shown to be optimal conditioned on popular conjectures [28, 1, 13].

In this paper, we study the problem of maintaining *Breadth First Search (BFS) Trees* (or shortest path trees for unweighted graphs) from a given source under dynamic updates. The applications of BFS trees range from network routing and distance estimation to social-influence analysis and web crawling, which fundamentally maintain BFS distances under a stream of edge updates. Given a graph $G(V, E)$ having n nodes and m edges, its BFS tree from any source $s \in V$ can be computed in $O(m + n)$ time. Recomputing the BFS tree from scratch after each update is thus prohibitive when updates arrive at high frequency on large networks. The classical result by Even and Shiloach [9] maintains a partially dynamic BFS tree in total $O(mn)$ or amortized $O(n)$ update time, a bound that remained the state of the art for directed graphs for decades. Later, Roditty and Zwick [28] showed that no combinatorial algorithm can improve the bound using the All Pairs Shortest Paths Conjecture. Further, it was shown that no polynomial improvement for partially dynamic BFS was possible using the combinatorial BMM conjecture by Abboud and Williams [1], even for offline updates (update sequence known in advance) [13]. However, in the worst case, the update time can still be $O(m)$. This was also shown to be tight by Henzinger et al. [13] using the Online Matrix Vector product (OMv) conjecture. For the fully dynamic setting, no algorithm is known to improve the trivial recomputation in $O(m)$ time. However, a fully dynamic version of ES trees was shown by Hanauer et al. [11] to perform well in practice despite having no better theoretical bounds.

We study the classical dynamic BFS tree problem in the prediction model. A motivating observation about real-world update sequences is that they are rarely adversarial: network changes tend to follow patterns, user behaviour is often repetitive, and future updates can frequently be anticipated from historical data or learned models. The emerging paradigm of *algorithms with predictions* (also called *learning-augmented algorithms*) formalises this intuition by equipping classical algorithms with a *prediction*—an auxiliary input, potentially produced by a machine-learning oracle. The goal is to determine if predictions enable provably faster algorithms while retaining correctness guarantees regardless of prediction quality. The key desired properties of such algorithms are *consistency* (optimal performance when predictions are perfect), *robustness* (no worse than the best prediction-free algorithms even for the worst prediction), and *smooth degradation* (performance deteriorates gracefully between the two extremes). In recent years, the area of algorithms with predictions has received significant attention, particularly for online problems and data structures (see [24, 25] for a survey), as well as for dynamic graph algorithms. Brand et al. [4] presented algorithms for dynamic transitive closure and approximate all pairs shortest paths, while McCauley et al. [23] presented algorithms for incremental topological ordering and cycle detection. Liu and Srinivas [22] presented a general framework applicable to a plethora of prob-

lems. Henzinger et al. [14] mostly focused on lower bounds conditioned on the OMv conjecture. Notably, these results employed different *error measures* to analyze their algorithms.

Related Work. An interesting variant of dynamic graph problems is the offline dynamic problems [8, 19, 26], where the update sequence is known in advance. This resembles the algorithms with predictions model for perfect predictions. In fact, most conditional lower bounds for dynamic graph problems also hold for this simpler variant [4].

A related problem for weighted graphs is the single source shortest path (SSSP), and its all source version: all pairs shortest path (APSP). For positively weighted graphs, King [20] extended the ES trees for partially dynamic SSSP to require $O(mnW)$ total time, where W is the maximum weight of an edge. However, fully dynamic SSSP cannot be solved faster than simply recomputing the solution from scratch, which also has a matching lower bound [13]. Incremental APSP can be solved in total $O(n^3W)$ update time [2], where W is the maximum weight of an edge. Decremental and fully dynamic APSP respectively requires total update time of $O(n^3S)$ and $O(mn^2 \log^4 n)$ [6], where S is the number of distinct edge weights.

Another related problem is the single source reachability (SSR), and its all source version: all pair reachability or transitive closure (TC). Incremental SSR and TC can be respectively maintained in $O(m)$ and $O(mn)$ total time [17]. Whereas, the decremental SSR is no better than decremental BFS, requiring $O(mn)$ total time [9]. Also, decremental TC can be maintained in total $O(mn)$ time [21, 27]. However, for fully dynamic updates, none of these problems improves over trivially recomputing from scratch, which is also matched by the corresponding lower bound [1]. Further, all these algorithms also require $\Omega(m)$ worst-case update time, having matching lower bounds [13].

Our Results

To analyze our proposed algorithms, we describe the error measures for the predictions. The vertices whose shortest distance from s is incorrectly predicted are called *error vertices*, with the corresponding difference called their *error*. The vertex prediction error η_v is the sum of degrees of *error vertices* (hence $\eta_v \leq m$), and the weighted vertex prediction error η_v^* is the *error weighted* sum of degrees of *error vertices* (hence $\eta_v^* \leq mn$). The edge prediction error η_e is the number of incorrectly predicted edge updates (hence $\eta_e \leq m$).

Our results can be summarized as follows:

Theorem 1.1 (Incremental BFS)**.** *Given a graph $G(V, E)$ having n vertices and m edges undergoing edge insertions with a predicted sequence of updates, the BFS tree can be reported in worst case $O(\eta_e+\eta_v)$ time using $O(mn)$ preprocessing time and space, with total update time $O(\eta_v^*+\eta_e)$.*

Theorem 1.2 (Decremental BFS)**.** *Given a graph $G(V, E)$ having n vertices and m edges undergoing edge deletions with a predicted sequence of updates, the BFS tree can be reported in worst case $O(\min\{m, \eta_e + \eta_v^*\})$ time using $O(mn)$ preprocessing time and space and total time $O(\eta_v^* + \eta_e)$.*

Remark 1.3. Given the polynomially tight lower bound using combinatorial BMM for partially dynamic BFS even for offline updates, we cannot hope to polynomially improve our preprocessing time.

Remark 1.4. These are smooth degradation of the ES Trees [9] requiring $O(m)$ time in the worst case, and total time $O(\eta_e + \eta_v^*) = O(mn)$.

Theorem 1.5 (Fully Dynamic BFS)**.** *Given a graph $G(V, E)$ having n vertices undergoing m edge insertions and deletions with a predicted sequence of updates, the BFS tree can be reported in worst case $O(\min\{m, \eta_e + \eta_v^*\})$ time using $O(m^2)$ preprocessing time and space.*

Remark 1.6. It is a smooth degradation of the trivial BFS algorithm requiring $O(m)$ time.

We also optimize the space utilization of our partially dynamic algorithm as follows.

Theorem 1.7 (Space Optimization for Incremental BFS)**.** *Given a graph $G(V, E)$ having n vertices and m edges undergoing edge insertions with a predicted sequence of updates, the BFS tree can be reported in worst case $O(\eta_e + \eta_v + n \log n)$ time using $O(mn)$ preprocessing time and $O(n^2)$ space, with total update time $O(\eta_v^* + \eta_e + n \log n)$.*

Theorem 1.8 (Space Optimization for Decremental BFS)**.** *Given a graph $G(V, E)$ having n vertices and m edges undergoing edge deletions with a predicted sequence of updates, the BFS tree can be reported in worst case $O(\eta_e + \eta_v^\star + nk)$ time using $O(mn)$ preprocessing time and $O\left(\frac{mn}{k} + n^2\right)$ space, for any $1 \leq k \leq n$, with total update time $O(\eta_v^* + \eta_e + nk)$.*

Hence, all our algorithms satisfy the desired properties of consistency, robustness, and smooth degradation. Additionally, we consider error correction mechanisms, which further improve the performance of our results. At its core, our algorithms can also be used as a batch update version of ES trees [9] for incremental, decremental, and fully dynamic settings, which may be of independent interest. Notably, this simple extension to batch updates is sufficient to give a prediction algorithm satisfying *robustness, consistency and smooth degradation*. Thus, our framework can potentially be generalized for other dynamic graph problems by considering their corresponding batch-update version.

Outline. We describe the notations and definitions used in the paper in Section 2, along with a brief description of the state of the art algorithms. Section 3 presents our incremental algorithm and Section 4 presents our decremental algorithm. Section 5 combines these into a fully dynamic framework. Section 6 develops the space-efficient incremental BFS algorithm. Section 7 develops the space-efficient storage approach for both decremental-only and fully dynamic BFS algorithms. The approach to error correction improving the performance of our algorithms is described in Section 8. Section 10 concludes with directions for future work.

2 Preliminaries

Let $G_0 = (V, E_0)$ denote the initial directed graph with vertex set V and edge set $E_0 \subseteq V \times V$, where $n = |V|$, $m_0 = |E_0|$ and a source $s \in V$. The graph is subject to an online sequence of edge updates $U = \{\langle e_1, t_1\rangle, \langle e_2, t_2\rangle, \ldots, \langle e_m, t_m\rangle\}$, where each e_i is an edge which is inserted ($t_i = +$) or deleted ($t_i = -$). We denote by $G_i = (V, E_i)$ the graph obtained after applying updates $\{\langle e_1, t_1\rangle, \cdots, \langle e_i, t_i\rangle\}$ to G_0. We denote a BFS tree of graph G_i from s as T_i. For each $v \in V$, we define the following:

- $\texttt{In}_i[v] = \{u \in V : (u, v) \in E_i\}$, the set of in-neighbours of v in G_i,
- $\texttt{Out}_i[v] = \{w \in V : (v, w) \in E_i\}$, the set of out-neighbours of v in G_i,
- $\texttt{deg}_i(v) = |\texttt{In}_i[v]| + |\texttt{Out}_i[v]|$, the degree of v in G_i.
- $\texttt{L}_i[v]$: the distance from s to v in T_i,
- $\texttt{Par}_i[v]$: the parent of v in T_i, with $\texttt{Par}_i[s] = \text{NULL}$.
- $\texttt{UP}_i[v] = \{u \in \texttt{In}_i[v] : \texttt{L}_i[u] = \texttt{L}_i[v] - 1\}$, the *upper parents*, or $\texttt{In}_i[v]$ at $\texttt{L}_i[v] - 1$ in T_i.

When the index i corresponding to G_i is clear from context, we drop the subscript i. This set of upper parents is only used by the decremental and fully dynamic algorithms to efficiently locate a replacement parent when the current parent edge is deleted.

2.1 Prediction Model and Error Measures

Alongside the real update sequence U, we have access to a *predicted* update sequence $\hat{U} = \{\langle \hat{e}_1, \hat{t}_1\rangle, \langle \hat{e}_2, \hat{t}_2\rangle, \ldots, \langle \hat{e}_m, \hat{t}_m\rangle\}$ provided by an external oracle (e.g., a learned model), which is available in full before any real update arrives. Thus, it can be preprocessed offline to improve the processing of real updates. Let $\hat{G}_i = (V, \hat{E}_i)$ denote the graph obtained by applying $\{\langle \hat{e}_1, \hat{t}_1\rangle, \cdots, \langle \hat{e}_i, \hat{t}_i\rangle\}$ to G_0, and let $\hat{T}_i$ be a BFS tree of $\hat{G}_i$ rooted at s. We store the corresponding arrays $\hat{\texttt{L}}_i[\cdot]$ and $\hat{\texttt{Par}}_i[\cdot]$ for all $\hat{G}_i$. For the decremental and fully dynamic algorithms, we also store $\hat{\texttt{UP}}_i[\cdot]$ for each $\hat{G}_i$. The predicted sequence $\hat{U}$ need not agree with U due to inaccurate predictions. Let the maximum exact prefix match of U and $\hat{U}$ be of i edges, i.e., $\hat{e}_k = e_k, \hat{t}_k = t_k$ for $k \leq i$ and at least $\hat{e}_{k+1} \neq e_{k+1}$ or $\hat{t}_{k+1} \neq t_{k+1}$. We thus define the following error measures for the current set $U = \{\langle e_1, t_1\rangle, \cdots, \langle e_j, t_j\rangle\}$ of $j \geq i$ updates.

Definition 2.1 (Edge Prediction Error η_e)**.** The number of real edge updates that were not correctly predicted starting from the first error $\eta_e = j - i$

Remark 2.2. This simple error measure is particularly significant for the partially dynamic updates, where a single incorrect update makes the entire remaining sequence unusable.

Remark 2.3. The simple η_e measure is a coarse upper bound on the length of divergent suffix. Error correction (Section 8) reduces from η_e the prefix where only order of updates differs.

Definition 2.4 (Vertex Prediction Error η_v)**.** The sum of degrees $\texttt{deg}(v)$ of vertices whose predicted level $\hat{\texttt{L}}_i[v]$ at last correct step i differs from the actual level $\texttt{L}_j[v]$ at step j

$$\eta_v \;=\; \sum_{v \in V : \hat{\texttt{L}}_i[v] \neq \texttt{L}_j[v]} \texttt{deg}(v)$$

Remark 2.5. Our erroneous prediction considers distance (instead of parent, etc.), making the error measure purely dependent on the graph and oblivious to the algorithm.

Remark 2.6. The degree of erroneously predicted vertex impacts the repair by affecting more incident edges, making η_v a cost-sensitive error measure rather than just a vertex-count error.

Definition 2.7 (Weighted Vertex Prediction Error η_v^*)**.** The weighted sum of degrees $\texttt{deg}(v)$ of vertices, with weight equivalent to error in prediction of level $|\hat{\texttt{L}}_i[v] - \texttt{L}_j[v]|$ from the last correct step i to the current step j.

$$\eta_v^* \;=\; \sum_{v \in V} \texttt{deg}(v) \left|\texttt{L}_j[v] - \hat{\texttt{L}}_i[v]\right|$$

Remark 2.8. The size of the distance error as a weight for erroneously predicted vertices captures the additional impact of the error, making it a suitable estimate for the update cost.

Why we use offline preprocessing. The predicted update sequence is available before the real updates arrive, so we can preprocess it offline and build the corresponding snapshots and auxiliary data structures in advance. This does not change the fact that the problem is online; rather, it shifts work away from the critical update path and allows the algorithm to exploit accurate predictions whenever they are available. Thus, even if the preprocessing cost is large, the offline phase is still justified because it reduces online latency and preserves correctness for every prediction quality.

2.2 State-of-the-art

We briefly summarize the classical ES-tree approach [9], which our algorithms extend.

Incremental BFS. When an edge (u, v) is inserted, if $\texttt{L}[u] + 1 < \texttt{L}[v]$, then the new edge offers a shorter path to v. The algorithm updates $\texttt{Par}[v] \leftarrow u$ and $\texttt{L}[v] \leftarrow \texttt{L}[u] + 1$, then propagates potential level decreases to the out-neighbours of v using a level-by-level queue, processing vertices in non-decreasing order of their new levels. Hence, whenever the level of a vertex v changes (decreases), we require $O(deg(v))$ time to process its neighbours. Since the level of a vertex can change $O(n)$ times throughout the algorithm, the total time required is $\sum_{v \in V} \texttt{deg}(v) \times O(n) = O(mn)$.

Decremental BFS. When an edge (u, v) is deleted, if (u, v) was v's parent edge (i.e., $u = \texttt{Par}[v]$), then v must find a replacement parent. The algorithm searches $\texttt{Up}[v]$ for an incoming neighbour at level $\texttt{L}[v] - 1$; if one exists it becomes the new parent of v and no further change is needed. Otherwise, v's level must increase, which may in turn invalidate the parent edges of

v's children, triggering a recursive repair. Whenever the level of a vertex v changes (increases), it updates $\texttt{Up}[v]$ and $\texttt{Up}[w]$ for each $(v, w) \in E$, requiring $O(\texttt{deg}(v))$ time. Vertices are processed level-by-level in non-decreasing order, ensuring that by the time a vertex is examined, all vertices at smaller levels have already been finalized. Again, as the level of a vertex can change $O(n)$ times throughout the algorithm, the total time required is $\sum_{v \in V} \texttt{deg}(v) \times O(n) = O(mn)$.

3 Incremental BFS with Predictions

We consider the incremental setting where $G_0(V, E_0)$ undergoes a sequence of edge insertions. We are given a predicted insertion sequence $\hat{U}$ and a real insertion sequence U, revealed online one edge at a time where all the $\hat{t}$'s are $+$. At step j, edge e_j is inserted into the graph, producing $G_j = (V, E_j)$.

3.1 Algorithmic Framework

The algorithm proceeds in two phases: an offline preprocessing phase over the predicted sequence, and an online phase that handles real updates. See Figure 1 for an example.

Preprocessing. We simulate the predicted insertions $\hat{e}_1, \ldots, \hat{e}_m$ in order, computing the sequence of predicted BFS trees $\hat{T}_0, \hat{T}_1, \ldots, \hat{T}_m$. For each i, we store the arrays $\hat{\texttt{L}}_i[\cdot]$ and $\texttt{P}\hat{\texttt{a}}\texttt{r}_i[\cdot]$ corresponding to $\hat{T}_i$. This phase runs entirely offline before any real update arrives.

Online Phase. We process the real insertions $e_1, \ldots, e_m$ one by one, maintaining the adjacency list $\texttt{Out}[\cdot]$ of the current graph and an integer i, initialised to 0. Intuitively, i tracks the latest step at which the real and predicted prefix sets coincide, so that $\hat{T}_i$ is a valid BFS tree of G_i.

At step j, after inserting e_j into $\texttt{Out}[\cdot]$, we distinguish two cases.

Case 1 (sequences agree). If $\{e_1, \ldots, e_j\} = \{\hat{e}_1, \ldots, \hat{e}_j\}$, then $\hat{T}_j$ is a valid BFS tree of G_j. We answer the query directly from the stored arrays $\hat{\texttt{L}}_j[\cdot]$ and $\texttt{P}\hat{\texttt{a}}\texttt{r}_j[\cdot]$, and set $i \leftarrow j$.

Case 2 (sequences diverge). Otherwise, the two sequences have diverged at or before step j. We load the stored snapshot $\hat{T}_i$, which is a valid BFS tree of G_i, and call BATCHINSERTEDGE with the unprocessed real edges $e_{i+1}, \ldots, e_j$ to repair the BFS tree. After answering the query from the updated arrays $\texttt{L}[\cdot]$ and $\texttt{Par}[\cdot]$, we roll back all temporary changes so that the maintained data structures revert to $\hat{T}_i$.

The pseudocode for BATCHINSERTEDGE is given in Algorithm 1.

3.2 Correctness

Lemma 3.1. *After* BATCHINSERTEDGE *processes all levels up to ℓ, the arrays $\texttt{L}[\cdot]$ and $\texttt{Par}[\cdot]$ are correct for every vertex whose distance from s in G_j is at most ℓ.*

Algorithm 1: batchInsertEdge($U_{i..j} = \{\langle e_{i+1}, t_{i+1}\rangle, \ldots, \langle e_j, t_j\rangle\}$)

Initialize empty lists $\mathcal{L}_{\mathrm{R}}[\ell]$ for all levels ℓ;
$\ell^* \leftarrow n$;
foreach $((u, v), +) \in U_{i..j}$ **do**
  add v to $\mathtt{Out}[u]$;
  if $\mathtt{L}[v] > \mathtt{L}[u] + 1$ **then**
    $\mathtt{Par}[v] \leftarrow u$;
    $\mathtt{L}[v] \leftarrow \mathtt{L}[u] + 1$;
    add v to $\mathcal{L}_{\mathrm{R}}[\mathtt{L}[v]]$;
    $\ell^* \leftarrow \min(\ell^*, \mathtt{L}[v])$;

for $\ell \leftarrow \ell^*$ **to** n **do**
  while $\mathcal{L}_{\mathrm{R}}[\ell]$ *not empty* **do**
    pop y from $\mathcal{L}_{\mathrm{R}}[\ell]$;
    if *vis[y]* **then**
      continue;
    vis[y] $\leftarrow$ true;
    foreach $z \in \mathtt{Out}[y]$ **do**
      if $\mathtt{L}[z] > \mathtt{L}[y] + 1$ **then**
        $\mathtt{L}[z] \leftarrow \mathtt{L}[y] + 1$;
        $\mathtt{Par}[z] \leftarrow y$;
        add z to $\mathcal{L}_{\mathrm{R}}[\mathtt{L}[z]]$;

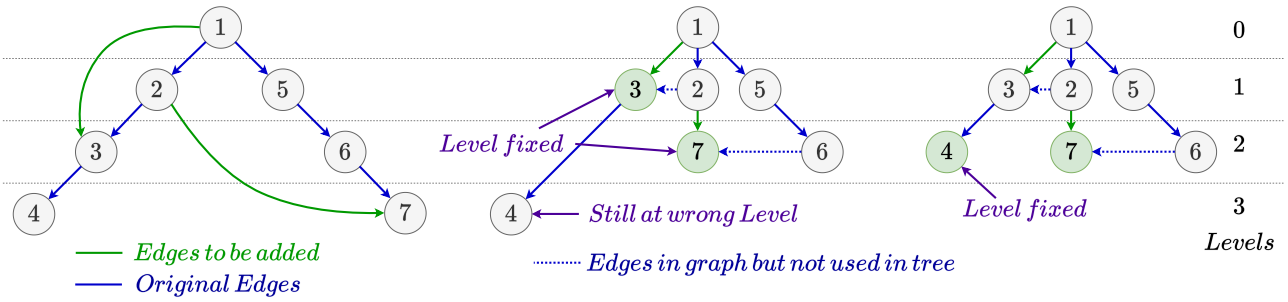


Figure 1: Incremental example. During initialization, nodes 3 and 7 move to $\mathcal{L}_{\mathrm{R}}[1]$ and $\mathcal{L}_{\mathrm{R}}[2]$ respectively and their levels are also fixed. The propagation pops node 3, looks at its children and fixes node 4 from level 3 to level 2 and also adds it in $\mathcal{L}_{\mathrm{R}}[2]$. Then at the next level, both 4 and 7 are popped and propagation ends.

Proof. We proceed by induction on ℓ, starting from ℓ^*, the smallest level at which any vertex is affected by the batch insertion.

Base case ($\ell = \ell^*$). Before the batch, the BFS tree is correct for all levels below ℓ^*. During initialisation, every vertex that can reach distance ℓ^* via a newly inserted edge is identified and assigned the correct level and parent. Hence all vertices at distance at most ℓ^* are correct after this step.

Inductive step. Suppose all vertices at distance at most ℓ are correct. Consider any vertex v with true distance $\ell + 1$ in G_j. Its shortest-path parent p satisfies $\texttt{L}[p] = \ell$ and $(p, v) \in e_j$. When p is processed at level ℓ, the edge (p, v) is relaxed, setting $\texttt{L}[v] \leftarrow \ell + 1$ and $\texttt{Par}[v] \leftarrow p$. No shorter distance is possible for v, since all vertices at levels below $\ell + 1$ are already finalised by the induction hypothesis. Hence all vertices at distance at most $\ell + 1$ are correct after processing level $\ell + 1$. □

3.3 Analysis

Preprocessing. We compute the required datastructures using the classical Incremental BFS algorithm, maintaining a copy of T_i and its associated structures after every update. This requires $O(mn)$ time and space.

Update BATCHINSERTEDGE. Each edge in the batch is processed once during the initialisation phase, contributing $O(\eta_e)$ work in total. Let $A \subseteq V$ denote the set of vertices whose level strictly decreases as a result of the batch insertion. For each $v \in A$, we scan all edges in $\texttt{Out}[v]$ once. The total running time is therefore

$$O\left(\eta_e + \sum_{v \in A} \texttt{deg}(v)\right) = O(\eta_e + \eta_v),$$

where the equality follows from Definition 2.4, since A is exactly the set of vertices whose level in G_j differs from their predicted level at step i. The space required is also bounded by $O(mn)$ due to previously built data structures.

Theorem 3.2. *Given a graph $G(V, E)$ having n vertices and m edges undergoing edge insertions with a predicted sequence of updates, the BFS tree can be reported in worst case $O(\eta_e + \eta_v)$ time using $O(mn)$ preprocessing time and space.*

Remark 3.3. The BFS tree T_j can also be obtained incrementally from T_{j-1} by inserting only e_j. The affected vertices in this one-step transition form a subset of those in the batch transition from $\hat{T}_i$ to T_j, so it is never more expensive. We use the batch form only because it extends naturally to the fully dynamic setting, while the actual update still runs in worst-case $O(\eta_e + \eta_v^*)$ time.

Theorem 1.1 (Incremental BFS)**.** *Given a graph $G(V, E)$ having n vertices and m edges undergoing edge insertions with a predicted sequence of updates, the BFS tree can be reported in worst case $O(\eta_e+\eta_v)$ time using $O(mn)$ preprocessing time and space, with total update time $O(\eta_v^*+\eta_e)$.*

4 Decremental BFS with Predictions

We consider the decremental setting in which $G_0 = (V, E_0)$ undergoes a sequence of edge deletions. We are given a predicted deletion sequence $\hat{U}$ and a real deletion sequence U, revealed online one edge at a time where all the $\hat{t}$'s are $-$. At step j, edge e_j is deleted from the graph, producing $G_j = (V, E_j)$. Our goal is to exploit agreement between $\hat{U}$ and U to accelerate BFS-tree maintenance.

4.1 Algorithmic Framework

The algorithm proceeds in two phases: an offline preprocessing phase over the predicted sequence, and an online phase that handles real updates. See Figure 2 for an example.

Preprocessing. We simulate the predicted deletions $\hat{e}_1, \ldots, \hat{e}_m$ in order, computing the sequence of predicted BFS trees $\hat{T}_0, \hat{T}_1, \ldots, \hat{T}_m$. For each j, we store the arrays $\hat{\texttt{L}}_j[\cdot]$, $\hat{\texttt{Par}}_j[\cdot]$, and the upper-parent lists $\texttt{UP}[\cdot]$ corresponding to $\hat{T}_j$. This phase runs entirely offline before any real update arrives.

Online Phase. We process the real deletions $e_1, \ldots, e_m$ one by one, maintaining the adjacency lists $\texttt{In}[\cdot]$ and $\texttt{Out}[\cdot]$ of the current graph and an integer i, initialised to 0. As in the incremental setting, i tracks the latest step at which the real and predicted prefix sets coincide, so that $\hat{T}_i$ is a valid BFS tree of G_i.

At step j, after deleting e_j from $\texttt{In}[\cdot]$ and $\texttt{Out}[\cdot]$, we distinguish two cases.

Case 1 (sequences agree). If $\{e_1, \ldots, e_j\} = \{\hat{e}_1, \ldots, \hat{e}_j\}$, then $\hat{T}_j$ is a valid BFS tree of G_j. We answer the query directly from the stored arrays $\hat{\texttt{L}}_j[\cdot]$ and $\hat{\texttt{Par}}_j[\cdot]$, and set $i \leftarrow j$.

Case 2 (sequences diverge). Otherwise, the two sequences have diverged at or before step j. We load the stored snapshot $\hat{T}_i$, which is a valid BFS tree of G_i, together with its arrays $\hat{\texttt{L}}_i[\cdot]$, $\hat{\texttt{Par}}_i[\cdot]$, and upper-parent lists $\texttt{UP}[\cdot]$. We then call BATCHDELETEEDGE with the unprocessed real edges $e_{i+1}, \ldots, e_j$ to repair the BFS tree. After answering the query from the updated arrays $\texttt{L}[\cdot]$ and $\texttt{Par}[\cdot]$, we roll back all temporary changes so that the maintained data structures revert to $\hat{T}_i$.

The pseudocode for BATCHDELETEEDGE is given in Algorithm 2.

4.2 Correctness

Lemma 4.1. *After* BATCHDELETEEDGE *processes all levels up to ℓ, the arrays $\texttt{L}[\cdot]$ and $\texttt{Par}[\cdot]$ are correct for every vertex whose distance from s in G_j is at most ℓ.*

Proof. We proceed by induction on ℓ, starting from ℓ^*, the smallest level at which any vertex is affected by the batch deletion.

Algorithm 2: batchDeleteEdge($U_{i..j} = \{\langle e_{i+1}, t_{i+1}\rangle, \ldots, \langle e_j, t_j\rangle\}$)

Initialize empty lists $\mathcal{L}_{\mathrm{L}}[\ell]$ for all levels ℓ;
$\ell^* \leftarrow n$;
PreprocessDeletions($U_{i\cdots j}$);
for $\ell \leftarrow \ell^*$ **to** n **do**
  RepairLevel(l);

PreprocessDeletions($U_{i\cdots j}$) :
foreach $((u, v), -) \in U_{i..j}$ **do**
  remove v from $\mathtt{Out}[u]$;
  remove u from $\mathtt{In}[v]$;
  if $u \in \mathit{UP}[v]$ **then**
    remove u from $\mathtt{UP}[v]$;
    if $\mathit{UP}[v] = \emptyset$ **then**
      add v to $\mathcal{L}_{\mathrm{L}}[\mathtt{L}[v]]$;
      $\ell^* \leftarrow \min(\ell^*, \mathtt{L}[v])$;
    else if $\mathit{Par}[v] = u$ **then**
      $\mathtt{Par}[v] \leftarrow \mathtt{UP}[v][0]$;

RepairLevel(l):
while $\mathcal{L}_{\mathrm{L}}[\ell]$ *not empty* **do**
  pop x from $\mathcal{L}_{\mathrm{L}}[\ell]$;
  if $\mathit{UP}[x] \neq \emptyset$ **then**
    $\mathtt{Par}[x] \leftarrow \mathtt{UP}[x][0]$;
    continue;
  $\mathtt{L}[x] \leftarrow \mathtt{L}[x] + 1$;
  add x to $\mathcal{L}_{\mathrm{L}}[\mathtt{L}[x]]$;
  $\mathtt{UP}[x] \leftarrow \emptyset$;
  foreach $p \in \mathit{In}[x]$ **do**
    if $\mathit{L}[p] = \mathit{L}[x] - 1$ **then**
      add p to $\mathtt{UP}[x]$;
  foreach $y \in \mathit{Out}[x]$ **do**
    if $x \notin \mathit{UP}[y]$ **then**
      continue;
    remove x from $\mathtt{UP}[y]$;
    if $\mathit{UP}[y] = \emptyset$ **then**
      add y to $\mathcal{L}_{\mathrm{L}}[\mathtt{L}[y]]$;
    else if $\mathit{Par}[y] = x$ **then**
      $\mathtt{Par}[y] \leftarrow \mathtt{UP}[y][0]$;

Base case ($\ell = \ell^*$). Before processing the batch, the BFS tree is correct up to level $\ell^* - 1$. During the initialisation pass, every vertex that directly lost all candidate parents at level $\ell^* - 1$ is enqueued in $\mathcal{L}_{\mathrm{L}}[\ell^*]$ and repaired; vertices that still retain at least one candidate in $\texttt{UP}[\cdot]$ are immediately assigned a valid parent. Hence all vertices at distance at most ℓ^* are correct after initialisation.

Inductive step. Assume all vertices at distance at most ℓ are correct. When we process level $\ell + 1$, each vertex v dequeued from $\mathcal{L}_{\mathrm{L}}[\ell + 1]$ either finds a valid parent in $\texttt{UP}[v]$ and is assigned one, or has $\texttt{UP}[v] = \varnothing$ and is moved to level $\ell + 2$ and re-enqueued. In the latter case, the children of v whose sole candidate parent was v are updated accordingly. Since all vertices at levels at most ℓ are already correct by the induction hypothesis, correctness extends to level $\ell + 1$, completing the induction. □

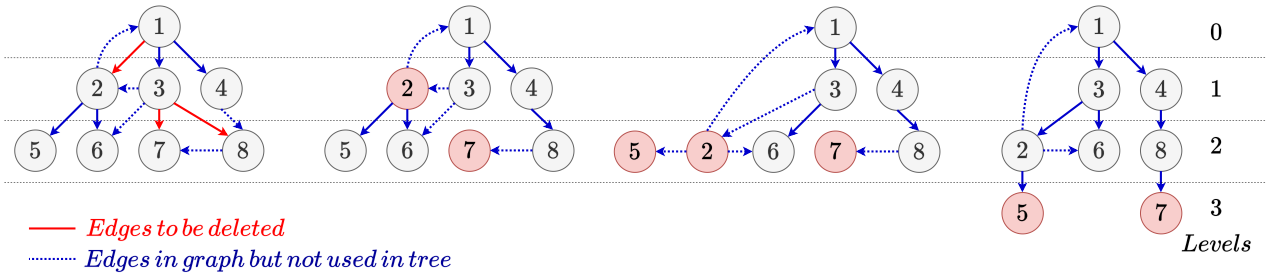


Figure 2: Decremental example. During initialization, nodes 2,7 and 8 lose their current parents but node 8 already has an extra node 4 in it's $\texttt{UP}[]$ so it reassigns to that directly while 2 and 7 are pushed to $\mathcal{L}_{\mathrm{L}}[1]$ and $\mathcal{L}_{\mathrm{L}}[2]$ respectively. During propagation, node 2 falls down while its children have it removed from their $\texttt{UP}[]$, 5 is pushed to $\mathcal{L}_{\mathrm{L}}[2]$ while 6 directly gets 3 from its $\texttt{UP}[]$. Then at the next level, 2 gets 3 as a parent but 5 and 7 fall down. Then at last level 5 and 7 get new parents.

4.3 Analysis

Preprocessing. We compute the required data structures using the classical Decremental BFS algorithm, maintaining a copy of T_i and its associated structures after every update. This requires $O(mn)$ time and space. However, we additionally maintain a copy of $\texttt{UP}_i[v]$ for each $\hat{G}_i$ requiring $O(m^2)$ time and space.

Update BATCHDELETEEDGE. Each edge in the batch is processed once during the initialisation phase, contributing $O(\eta_e)$ work in total. Let $A \subseteq V$ denote the set of vertices whose level strictly increases as a result of the batch deletion. For each $v \in A$, we scan all edges in $\texttt{In}[v]$ and $\texttt{Out}[v]$ once per level increase of v. The total running time is therefore

$$O\left(\eta_e + \sum_{v \in A} \deg(v) \cdot |\mathrm{L}_j[v] - \hat{\mathrm{L}}_i[v]|\right) \subseteq O\left(\eta_e + \sum_{v \in V} \deg(v) \cdot |\mathrm{L}_j[v] - \hat{\mathrm{L}}_i[v]|\right) = O(\eta_e + \eta_v^*),$$

where the last equality follows from Definition 2.7. The space required is also bound by the data structures built during preprocessing.

Remark 4.2. In the worst case each vertex's level may increase by $O(n)$, so $\eta_v^* = O(n \sum_{v \in V} \deg(v)) = O(nm)$. When $\eta_e + \eta_v^*$ exceeds $O(m)$, we fall back to rebuilding the BFS tree from scratch in $O(m)$ time (similar approach was previously used by [11]).

Theorem 4.3. *Given a graph $G(V, E)$ having n vertices and m edges undergoing edge deletions with a predicted sequence of updates, the BFS tree can be reported in worst case $O(\min\{m, \eta_e + \eta_v^*\})$ time using $O(m^2)$ preprocessing time and space.*

Remark 4.4. The BFS tree T_j can be obtained from T_{j-1} by deleting only e_j. The affected vertices in this one-step transition are a subset of those in the batch transition from $\hat{T}_i$ to T_j, so it is never more expensive. We use the batch form only because it extends naturally to the fully dynamic setting, while the actual update still runs in worst-case $O(\eta_e + \eta_v^*)$ time.

4.4 Pointer-Based Decremental Storage

In the decremental setting, we can replace $\texttt{UP}[\cdot]$ by one cursor per vertex in $\texttt{In}[v]$. Thus, instead of storing full upper-parent lists, we store $\texttt{ptr}[j] = (\texttt{ptr}_j[1], \ldots, \texttt{ptr}_j[n])$, where $\texttt{ptr}_j[v]$ is the current scan position at step j.

After deleting (u, v), if it is not the parent edge of v, only the adjacency lists change. Otherwise, we scan forward from $\texttt{ptr}[v]$ to find a valid parent at level $\texttt{L}[v] - 1$; if none exists, we increase $\texttt{L}[v]$, enqueue v, and reset $\texttt{ptr}[v]$. This stores one pointer per vertex per snapshot, using $O(n)$ space per update and $O(mn)$ overall, and reproduces the same parent-selection behavior as $\texttt{UP}[\cdot]$.

Remark 4.5. The $\texttt{UP}[\cdot]$ formulation is kept for consistency with the fully dynamic framework. For decremental BFS alone, the pointer-based version is simpler and more space-efficient, so it is preferred.

Theorem 1.2 (Decremental BFS)**.** *Given a graph $G(V, E)$ having n vertices and m edges undergoing edge deletions with a predicted sequence of updates, the BFS tree can be reported in worst case $O(\min\{m, \eta_e + \eta_v^*\})$ time using $O(mn)$ preprocessing time and space and total time $O(\eta_v^* + \eta_e)$.*

5 Fully Dynamic BFS with Predictions

We consider the fully dynamic setting in which $G_0 = (V, E_0)$ undergoes a sequence of edge insertions and deletions. We are given a predicted update sequence $\hat{U}$ and a real update sequence U, revealed online one update at a time. At step j, update e_j is applied to the graph, producing $G_j = (V, E_j)$. Our goal is to exploit agreement between $\hat{U}$ and U to accelerate BFS-tree maintenance.

5.1 Algorithmic Framework

The algorithm proceeds in two phases: an offline preprocessing phase over the predicted sequence, and an online phase that handles real updates. See Figure 3 for an example.

Preprocessing. We simulate the predicted updates $\hat{e}_1, \ldots, \hat{e}_m$ in order, computing the sequence of predicted BFS trees $\hat{T}_0, \hat{T}_1, \ldots, \hat{T}_m$. For each j, we store the arrays $\hat{\texttt{L}}_j[\cdot]$, $\hat{\texttt{Par}}_j[\cdot]$, and the upper-parent lists $\texttt{UP}[\cdot]$ corresponding to $\hat{T}_j$. This phase runs entirely offline before any real update arrives.

Online Phase. We process the real updates $e_1, \ldots, e_m$ one by one, maintaining the adjacency lists $\texttt{In}[\cdot]$ and $\texttt{Out}[\cdot]$ of the current graph and an integer i, initialised to 0. As before, i tracks the latest step at which the real and predicted prefix sets coincide, so that $\hat{T}_i$ is a valid BFS tree of G_i.

At step j, we first apply update (u, v, t) to the adjacency lists: if $t = +$ we insert (u, v) into $\texttt{In}[\cdot]$ and $\texttt{Out}[\cdot]$; if $t = -$ we delete (u, v) from $\texttt{In}[\cdot]$ and $\texttt{Out}[\cdot]$. We then distinguish two cases.

Case 1 (sequences agree). If $\{e_1, \ldots, e_j\} = \{\hat{e}_1, \ldots, \hat{e}_j\}$, then $\hat{T}_j$ is a valid BFS tree of G_j. We answer the query directly from the stored arrays $\hat{\texttt{L}}_j[\cdot]$ and $\hat{\texttt{Par}}_j[\cdot]$, and set $i \leftarrow j$.

Case 2 (sequences diverge). Otherwise, the two sequences have diverged at or before step j. We load the stored snapshot $\hat{T}_i$, together with its arrays $\hat{\texttt{L}}_i[\cdot]$, $\hat{\texttt{Par}}_i[\cdot]$, and upper-parent lists $\texttt{UP}[\cdot]$. Let E_{ins} and E_{del} denote the insertions and deletions respectively among the unprocessed real updates $e_{i+1}, \ldots, e_j$. We call BATCHDYNAMICUPDATE$(E_{\text{del}}, E_{\text{ins}})$ to repair the BFS tree. After answering the query from the updated arrays $\texttt{L}[\cdot]$ and $\texttt{Par}[\cdot]$, we roll back all temporary changes so that the maintained data structures revert to $\hat{T}_i$.

The pseudocode for BATCHDYNAMICUPDATE is given in Algorithm 3.

Algorithm 3: batchDynamicUpdate$(E_{\text{del}}, E_{\text{ins}})$

Initialize empty lists $\mathcal{L}_{\text{L}}[\ell], \mathcal{L}_{\text{R}}[\ell]$ for all levels ℓ;
$\ell^* \leftarrow n$;
PreprocessDeletions(E_{del});
foreach $(u, v) \in E_{\text{ins}}$ **do**
  add v to $\texttt{Out}[u]$;
  add u to $\texttt{In}[v]$;
  if $\texttt{L}[v] > \texttt{L}[u] + 1$ **then**
    $\texttt{Par}[v] \leftarrow u$;
    $\texttt{L}[v] \leftarrow \texttt{L}[u] + 1$;
    $\texttt{UP}[v] \leftarrow \{u\}$;
    add v to $\mathcal{L}_{\text{R}}[\texttt{L}[v]]$;
    $\ell^* \leftarrow \min(\ell^*, \texttt{L}[v])$;
  else if $\texttt{L}[v] = \texttt{L}[u] + 1$ **then**
    add u to $\texttt{UP}[v]$;
for $\ell \leftarrow \ell^*$ **to** n **do**
  RepairLevel(l);
  while $\mathcal{L}_{\text{R}}[\ell]$ *not empty* **do**
    pop y from $\mathcal{L}_{\text{R}}[\ell]$;
    foreach $z \in \texttt{Out}[y]$ **do**
      if $z \in \mathcal{L}_{\text{L}}[l + 1]$ **then**
        $\texttt{Par}[z] \leftarrow y$;
        remove z from $\mathcal{L}_{\text{L}}[l + 1]$;
      else if $\texttt{L}[z] > \texttt{L}[y] + 1$ **then**
        $\texttt{L}[z] \leftarrow \texttt{L}[y] + 1$;
        $\texttt{Par}[z] \leftarrow y$;
        $\texttt{UP}[z] \leftarrow \{y\}$;
        add z to $\mathcal{L}_{\text{R}}[\texttt{L}[z]]$;
      else if $\texttt{L}[z] = \texttt{L}[y] + 1$ **then**
        add y to $\texttt{UP}[z]$;

5.2 Correctness

Lemma 5.1. *After* BATCHDYNAMICUPDATE *sweeps levels from ℓ^* up to ℓ, the arrays $\texttt{L}[\cdot]$ and $\texttt{Par}[\cdot]$ are correct for every vertex whose distance from s in G_j is at most ℓ.*

Proof. We proceed by induction on ℓ, starting from ℓ^*, the smallest level at which any vertex is affected by the batch update.

Base case ($\ell = \ell^*$). Before applying the batch $E_{\text{del}}, E_{\text{ins}}$, the BFS tree is correct up to level $\ell^* - 1$. The initialisation pass processes deletions before insertions, enqueuing and fixing every vertex whose parent or candidate-parent set changed directly at level ℓ^*. Hence all vertices at distance at most ℓ^* are correct after the initialisation pass.

Inductive step. Assume all vertices at distance at most ℓ are correct. When processing level $\ell + 1$, we first handle deletion-driven level increases: each such vertex either finds a candidate parent at level ℓ and is assigned one, or is raised to the next level and re-enqueued with its children updated accordingly. We then handle insertion-driven level decreases: these relax outgoing edges and assign parent pointers for vertices that newly obtain distance $\ell + 1$. Since deletions are processed before insertions at each level, and all vertices at levels at most ℓ are already correct by the induction hypothesis, every vertex whose true distance is $\ell + 1$ receives a correct parent and distance during this sweep. Hence correctness extends to level $\ell + 1$, completing the induction. □

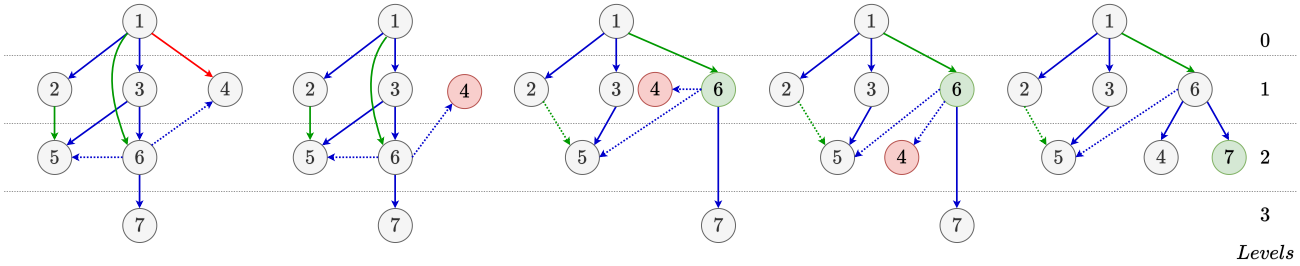


Figure 3: Fully dynamic example. During initialization, node 4 loses it's current parent and is pushed to $\mathcal{L}_{\text{L}}[1]$, meanwhile 2 is pushed in $\texttt{UP}[]$ of 5 and node 6 is pushed to $\mathcal{L}_{\text{R}}[1]$. During propagation, node 4 falls down and goes into $\mathcal{L}_{\text{L}}[2]$, while 6 is popped from $\mathcal{L}_{\text{R}}[1]$ and fixes level of both 4 and 7.

5.3 Analysis

Preprocessing. We compute the required data structures using the trivial BFS algorithm, maintaining a copy of T_i and $\texttt{UP}_i[v]$ for each $\hat{G}_i$, requiring $O(m^2)$ time and space.

Update BATCHDYNAMICUPDATE**.** Each edge in E_{ins} and E_{del} is processed once during the initialisation phase, contributing $O(\eta_e)$ work in total. Let $A_1 \subseteq V$ denote the set of vertices whose level strictly decreases due to batch insertions, and let $A_2 \subseteq V$ denote the set of vertices whose

level strictly increases due to batch deletions. For each $v \in A_1$ we scan $\texttt{Out}[v]$ once; for each $v \in A_2$ we scan $\texttt{In}[v]$ and $\texttt{Out}[v]$ once per level increase. The total running time is therefore

$$O\left(\eta_e + \sum_{v \in A_1} \texttt{deg}(v) + \sum_{v \in A_2} \texttt{deg}(v) \cdot |\texttt{L}_j[v] - \hat{\texttt{L}}_i[v]|\right)$$
$$\subseteq O\left(\eta_e + \sum_{v \in V} \texttt{deg}(v) \cdot |\texttt{L}_j[v] - \hat{\texttt{L}}_i[v]|\right) = O(\eta_e + \eta_v^*),$$

where the inclusion holds because each term in the A_1 sum is bounded by the corresponding weighted term (the level difference is at least 1), and the last equality follows from Definition 2.7. The space required is again bounded by the size of data structures built during preprocessing.

Remark 5.2. In the worst case each vertex level may change by $O(n)$, so $\eta_v^* = O(n \sum_{v \in V} \texttt{deg}(v)) = O(nm)$. When $\eta_e + \eta_v^*$ exceeds $O(m)$, we fall back to rebuilding the BFS tree from scratch in $O(m)$ time (similar approach was previously used by [11]).

Theorem 1.5 (Fully Dynamic BFS)**.** *Given a graph $G(V, E)$ having n vertices undergoing m edge insertions and deletions with a predicted sequence of updates, the BFS tree can be reported in worst case $O(\min\{m, \eta_e + \eta_v^*\})$ time using $O(m^2)$ preprocessing time and space.*

Remark 5.3. Unlike the incremental and decremental settings, processing the single update e_j directly on T_{j-1} is not necessarily better than the batch transition from $\hat{T}_i$ to T_j. Insertions and deletions within a batch may offset one another, resulting in fewer net level changes than would arise from processing each update individually. The batch formulation is therefore the natural choice for the fully dynamic setting as otherwise the worst case complexity is not bounded accordingly, losing the property of smooth degradation of the algorithm.

6 Space-Efficient Incremental BFS Algorithm

To use predictions, we must store the sequence of predicted BFS trees $\hat{T}_0, \hat{T}_1, \ldots, \hat{T}_m$. Naive storage requires $O(mn)$ space if we keep $\hat{\texttt{Par}}_j[\cdot]$ and $\hat{\texttt{L}}_j[\cdot]$ for all n vertices at all m steps. In the incremental setting, we can exploit the fact that BFS levels only decrease to compress this history to $O(n^2)$ space.

Data structure. For each vertex $v \in V$, maintain a chronological list of the steps at which its parent or level changes:

$$\mathrm{data}[v] = \big\{(p_1^{(v)}, d_1^{(v)}, t_1^{(v)}),\ (p_2^{(v)}, d_2^{(v)}, t_2^{(v)}),\ \ldots\big\}.$$

Each tuple (p, d, t) records that at predicted step t the parent of v became p and its level became d. If no tuple satisfies $t \le j$, then the values from $\hat{T}_0$ apply.

Space bound. In the incremental-only model, every level change of a vertex is a strict decrease. Since BFS levels are integers in $\{0, 1, \ldots, n-1\}$, each vertex can change parent and level at most $n-1$ times. Therefore the total number of stored tuples is at most $n(n-1)$, which gives $O(n^2)$ space.

Access cost. To recover the state of a single vertex at step j, we binary-search $\mathrm{data}[v]$ for the last tuple with timestamp at most j. This costs $O(\log k_v)$ time, where $k_v = |\mathrm{data}[v]| \le n-1$, hence $O(\log n)$ in the worst case. Reconstructing one full snapshot naively costs $O(n \log n)$.

Theorem 1.7 (Space Optimization for Incremental BFS)**.** *Given a graph $G(V, E)$ having n vertices and m edges undergoing edge insertions with a predicted sequence of updates, the BFS tree can be reported in worst case $O(\eta_e + \eta_v + n \log n)$ time using $O(mn)$ preprocessing time and $O(n^2)$ space, with total update time $O(\eta_v^* + \eta_e + n \log n)$.*

7 Space-Efficient Decremental BFS Algorithm

To use predictions, we must store the sequence of predicted BFS trees $\hat{T}_0, \hat{T}_1, \ldots, \hat{T}_m$. Naively storing full trees requires $O(mn)$ space but we plan to optimize it.

7.1 Per-Vertex Change Lists

For each vertex $v \in V$, maintain a chronological list

$$\mathrm{data}[v] = \big\{(p_1^{(v)}, d_1^{(v)}, t_1^{(v)}),\ (p_2^{(v)}, d_2^{(v)}, t_2^{(v)}),\ \ldots\big\},$$

where each tuple records that at predicted step t the parent of v became p and its level became d. When no tuple has timestamp at most j, the state from $\hat{T}_0$ applies.

In the decremental and fully dynamic settings, this list can be interpreted together with a cursor into $\texttt{In}[v]$. When a candidate parent becomes invalid, the cursor advances through $\texttt{In}[v]$ until a valid parent at level $\texttt{L}[v]-1$ is found. If no such parent exists, then $\texttt{L}[v]$ increases by one and v is placed in the lowering queue.

7.2 Space Reduction by Skipping Small Pointer Moves

To save space, we do not store every cursor movement. Fix an integer $k \geq 1$ and $k \leq n$, and store a new tuple for v only when the cursor has advanced by at least k positions in $\texttt{In}[v]$ since the previous stored tuple. Equivalently, we keep only every k-th relevant state of the scan.

For a fixed vertex v and a fixed level, this stores at most $\lceil \texttt{deg}(v)/k \rceil \leq \texttt{deg}(v)/k + 1$ tuples. Since a vertex can change level at most $O(n)$ times, the total storage becomes

$$O\left(\sum_{v \in V} n(\frac{\texttt{deg}(v)}{k} + 1)\right) = O\Big(\frac{mn}{k} + n^2\Big)\,.$$

Thus, the space drops by a factor of k compared with storing every cursor state.

7.3 Reconstructing a Snapshot

To recover the tree at step i, we load the latest stored state before i and replay at most $k-1$ skipped cursor moves per affected vertex. Hence the reconstruction cost increases by an additive factor of at most $O(k)$ per vertex whose exact state was not stored, giving worst case cost of $O(\eta_e + \eta_v^* + nk)$.

7.4 Space–Time Trade-off Parameterized by k

k	**Space**	**Reconstruction Cost**	**Interpretation**
1	$O(mn + n^2)$	$O(\eta_e + \eta_v^* + n)$	Full storage
$\sqrt{m}$	$O(n\sqrt{m} + n^2)$	$O(\eta_e + \eta_v^* + n\sqrt{m})$	Balanced
n	$O(m + n^2)$	$O(\eta_e + \eta_v^* + n^2)$	Minimal storage — maximum reconstruction time

Table 1: Space–time trade-off in the decremental setting as a function of k. Increasing k reduces storage but increases the amount of work needed to reconstruct a snapshot.

Theorem 1.8 (Space Optimization for Decremental BFS)**.** *Given a graph $G(V, E)$ having n vertices and m edges undergoing edge deletions with a predicted sequence of updates, the BFS tree can be reported in worst case $O(\eta_e + \eta_v^\star + nk)$ time using $O(mn)$ preprocessing time and $O\big(\frac{mn}{k} + n^2\big)$ space, for any $1 \leq k \leq n$, with total update time $O(\eta_v^* + \eta_e + nk)$.*

8 Error Correction

We present error correction for the fully dynamic setting, where trivial error correction also works for partially dynamic setting. The repair cost of BATCHDYNAMICUPDATE depends on η_e and η_v^* relative to $\hat{T}_i$. When the real and predicted batches share a common subset of updates M, we can start the repair from the later snapshot $\hat{T}_k$ instead of $\hat{T}_i$, where $k > i$.

Setup. From the divergence point (first error between U and $\hat{U}$) i and a update step $j > i$. Let $U_{i..j} = \{\langle e_{i+1}, t_{i+1}\rangle, \ldots, \langle e_j, t_j\rangle\}$ and $\hat{U}_{i..j} = \{\langle \hat{e}_{i+1}, \hat{t}_{i+1}\rangle, \ldots, \langle \hat{e}_j, \hat{t}_j\rangle\}$ denote the real and predicted batch updates.

8.1 Trivial Error Correction

If at some point $U_{i..j} = \hat{U}_{i..j}$, i.e., the set of updates are the same though their order might differ. Then $\hat{T}_j$ is a valid BFS tree of G_j as both batches contain the same (edge, type) pairs. Hence, we reset the value of i to j for all the future updates.

8.2 Non-Trivial Error Correction

When the batches are not set-equivalent, they may still share a large common subset. We now describe the error correction mechanisms for our algorithms.

Partially Dynamic

Let M be the largest prefix of $\hat{U}_{i...j}$ present in $U_{i...j}$, say $\hat{U}_{i...k}$. This can easily be computed in $O(\eta_e)$ time. Clearly, M is the set of matched updates, and define the unmatched sets

$$A = U_{i..j} \setminus M.$$

Since the matched updates M were applied to both G_j and $\hat{G}_k$, the two graphs differ only in A. Hence, in the new batch update we can simply use $U_{k...j}$ instead of the previous $U_{i...j}$. Thus, we get the following:

Theorem 8.1. BATCHINSERTEDGE$(U_{k...j})$ *from* $\hat{T}_k$ *yields a valid BFS tree of* G_j *in worst-case time* $O(\tilde{\eta}_e + \tilde{\eta}_v)$ *after error correction cost of* $O(\eta_e)$*, where* $\tilde{\eta}_e = |U_{i..j}| - |M|$ *and* $\tilde{\eta}_v$ *is the vertex error measured relative to* $\hat{T}_k$*.*

Theorem 8.2. BATCHDELETEEDGE$(U_{k...j})$ *from* $\hat{T}_k$ *yields a valid BFS tree of* G_j *in worst-case time* $O(\tilde{\eta}_e + \tilde{\eta^*}_v)$ *after error correction cost of* $O(\eta_e)$*, where* $\tilde{\eta}_e = |U_{i..j}| - |M|$ *and* $\tilde{\eta^*}_v$ *is the weighted vertex error measured relative to* $\hat{T}_k$*.*

Remark 8.3. The corrected edge and vertex error satisfies $\tilde{\eta}_e \leq \eta_e$, $\tilde{\eta}_v \leq \eta_v$ and $\tilde{\eta^*}_v \leq \eta_v^*$ as the change is level of vertices is monotonous.

Fully Dynamic

For any k, let $M = U_{i..j} \cap \hat{U}_{i..k}$ be the matched updates, and define the unmatched sets

$$P = \hat{U}_{i..k} \setminus M, \qquad A = U_{i..j} \setminus M.$$

Since the matched updates M were applied to both G_j and $\hat{G}_k$, the two graphs differ only in $P \cup A$. Starting from $\hat{T}_k$, we construct a corrected batch that transforms $\hat{G}_k$ into G_j:

$$E^*_{\text{del}} = \big\{(u,v) : (u,v,+) \in P\big\} \cup \big\{(u,v) : (u,v,-) \in A\big\},$$
$$E^*_{\text{ins}} = \big\{(u,v) : (u,v,-) \in P\big\} \cup \big\{(u,v) : (u,v,+) \in A\big\}.$$

Computing the value of k that minimizes $|E^*_{\text{del}}| + |E^*_{\text{ins}}|$ can be easily done in $O(\eta_e)$ time. Unmatched predicted insertions are reversed (deleted), unmatched predicted deletions are reversed (inserted), and unmatched actual updates are applied directly. We then call BATCHDYNAMICUPDATE$(E^*_{\text{del}}, E^*_{\text{ins}})$ from $\hat{T}_k$.

Theorem 8.4. BATCHDYNAMICUPDATE$(E^*_{\text{del}}, E^*_{\text{ins}})$ *from $\hat{T}_k$ yields a valid BFS tree of G_j in worst-case time $O(\min\{m,\ \tilde{\eta}_e + \tilde{\eta}^*_v\})$ after error correction cost of $O(\eta_e)$, where*

$$\tilde{\eta}_e = |U_{i..j}| + |\hat{U}_{i..k}| - 2|M|$$

and $\tilde{\eta}^*_v$ is the weighted vertex error measured relative to $\hat{T}_k$.

Remark 8.5. The corrected edge error satisfies $\tilde{\eta}_e \le \eta_e$, so the batch size never increases. However, $\tilde{\eta}^*_v$ may be larger or smaller than η^*_v depending on whether the matched updates M moved predicted levels closer to or further from the true distances in G_j. Non-trivial error correction is therefore most beneficial when $|M|$ is large and the updates in M are level-stabilising.

8.3 Effect on Space-Efficient Algorithms

After each error correction, an additional cost of $O(n \log n)$ and $O(nk)$ is incurred in the incremental and decremental cases, respectively, for the space-optimized variant. However, we can further improve the total extra cost paid for the incremental case.

Efficient search for increasing-time queries. When queries arrive in increasing time order, we maintain a pointer into $\mathrm{data}[v]$ for each v and advance it using exponential (doubling) jumps $(+1, +2, +4, \dots)$ until we reach or overshoot the next query time. The overshoot is resolved by a binary search over the final interval. For a monotone query sequence, each stored entry is skipped only a constant number of times amortized, so the total work over all queries is $O(\sum_v k_v) = O(n^2)$.

Thus, instead of paying $O(n \log n)$ after each potential $O(m)$ error corrections, we only require total $O(n^2)$ time.

9 Experimental Evaluation

We study how prediction error affects the running time of our algorithms. The graphs used in the experiments were downloaded from `konnect.cc`. Unless otherwise stated, the experiments were conducted on graphs with $n = 1000$ vertices and $m = 100000$ updates.

To generate test cases with a controlled error rate, let T be the total number of updates. For an error rate of $x\%$, we create a vector of length T and randomly place 1s in $(100 - x)\%$ of the positions, while the remaining positions are set to 0. At every position where the vector contains a 1, the prefix set of predicted updates must match the prefix set of actual updates at that point, up to reordering. This definition aligns with our error measures. Otherwise purely random test case generation will be hard for measuring erro rate.

9.1 Fully Dynamic Setting

Figure 4 shows the fully dynamic algorithm with and without error correction, along with the classical baseline. When the prediction error is small, both predicted variants are much faster than the classical method. As the error rate increases, the runtime rises steadily, and a sharp increase is seen near error rate 1.0. This is expected, since larger prediction errors reduce the benefit of the predicted information and require more work to repair it.

The version with error correction performs better than the version without error correction for moderate and high error rates. This shows that correction helps recover useful structure from partially correct predictions. When the error becomes very large, the benefit of prediction decreases, and the runtime moves closer to the cost of handling inaccurate batches. The classical baseline stays relatively stable across all error rates, but it is slower than the prediction-based methods when the prediction is reasonably accurate.

9.2 Incremental and Decremental Settings

Figure 5 shows the results for the incremental and decremental settings, each with three variants: no error correction, with error correction, and classical. The incremental algorithm benefits clearly from prediction when the error rate is low. For small to moderate error rates, the predicted versions stay close to each other, while the classical method becomes faster only when prediction quality drops significantly. At higher error rates, the incremental runtime increases more noticeably, but the corrected version remains more stable than the uncorrected one.

The decremental setting follows a similar pattern, although the runtime increases more sharply as the error rate grows. The corrected decremental algorithm consistently performs better than or close to the uncorrected variant, showing that error correction reduces the effect of inaccurate predictions. The classical decremental baseline remains stable, but it does not benefit from prediction.

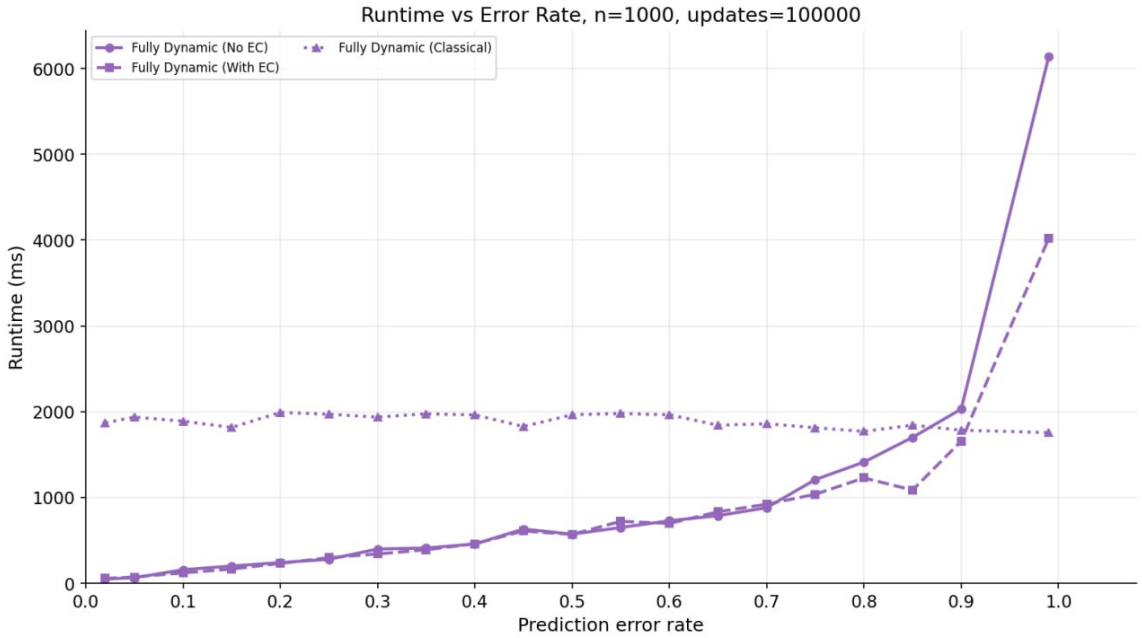


Figure 4: Runtime of the fully dynamic algorithm versus prediction error rate for $n = 1000$ and 100000 updates.

9.3 Discussion

These experiments show that prediction is most useful when the forecast is reasonably accurate. In that case, the runtime is much lower than the classical baseline. Error correction improves robustness, especially in the fully dynamic and decremental settings. As the prediction error increases, the advantage of prediction gradually decreases, which is consistent with the dependence of the running time on the error measures η_e, η_v, and η_v^*.

Overall, the results show that prediction-assisted BFS maintenance can give strong speedups in practice, while error correction helps retain these gains even when the predicted update sequence is imperfect.

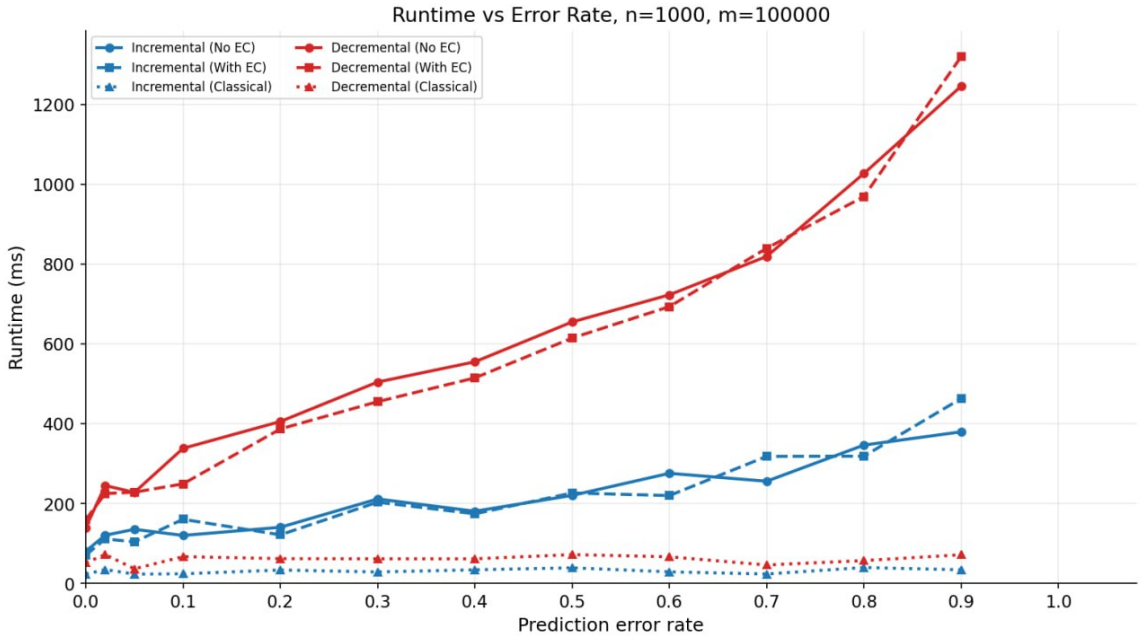


Figure 5: Runtime of the incremental and decremental algorithms versus prediction error rate for $n = 1000$ and $m = 100000$.

10 Conclusion

We have presented algorithms using the prediction models for maintaining a BFS tree in a directed dynamic graph, covering the incremental, decremental, and fully dynamic settings. Our algorithms exploit a predicted update sequence, which are preprocessed offline to achieve running times that scale with prediction quality rather than worst-case graph parameters. Specifically, the update time is $O(1)$ when predictions are perfect, $O(\eta_e + \eta_v)$ for incremental updates, and $O(\min(\eta_e + \eta_v^*,\, m))$ for decremental and fully dynamic updates — never exceeding the classical $O(m)$ bound regardless of prediction quality. Moreover, the total update time for both incremental and decremental algorithms is also bounded by $O(\min(\eta_e+\eta_v^*,\, m))$ - never exceeding the classical $O(mn)$ total time of ES Trees [9]. Thus, our algorithms satisfy the desired properties of consistency, robustness, and smooth degradation, of the algorithms with prediction model. We also presented a space-efficient storage schemes for the incremental algorithm that reduce the naive $O(mn)$ space to $O(n^2)$ at the expense of additional time complexity of $O(n \log n)$ in worst case and total update time. Similarly, we presented a space-efficient storage scheme for the decremental algorithm that reduces the $O(mn)$ space to $O(mn/k)$ at the expense of additional time complexity of $O(nk)$ in worst case and total update time. Finally, we presented some trivial and non-trivial error correcting mechanisms to reduce the error measures on which our algorithms are dependent.

Several directions remain open for future work. Firstly, to implement a full benchmark suite to evaluate our algorithms on real dynamic graph datasets and measure the percentage improvement in update time over classical approaches, particularly with respect to prediction error measures. On the theoretical side, it would be interesting to study *adaptive* prediction models that are refined online as real updates arrive, potentially tightening the error measures over time. Finally, our algorithms at its core are simple extensions of the classical ES Trees for handling batch up-

dates, which proves sufficient to give a prediction algorithm satisfying *robustness, consistency, and smooth degradation*. Thus, our framework may potentially be generalized for other dynamic graph problems using their corresponding batch update version.

References

[1] Amir Abboud and Virginia Vassilevska Williams. Popular conjectures imply strong lower bounds for dynamic problems. In *55th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2014, Philadelphia, PA, USA, October 18-21, 2014*, pages 434–443. IEEE Computer Society, 2014.

[2] Giorgio Ausiello, Giuseppe F. Italiano, Alberto Marchetti-Spaccamela, and Umberto Nanni. Incremental algorithms for minimal length paths. *J. Algorithms*, 12(4):615–638, 1991.

[3] Aaron Bernstein, Maximilian Probst Gutenberg, and Christian Wulff-Nilsen. Decremental strongly connected components and single-source reachability in near-linear time. *SIAM J. Comput.*, 52(on):STOC19–128–STOC19–155, 2019.

[4] Jan van den Brand, Sebastian Forster, Yasamin Nazari, and Adam Polak. On dynamic graph algorithms with predictions. In *Proceedings of the 2024 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 3534–3557. SIAM, 2024.

[5] Julia Chuzhoy, Yu Gao, Jason Li, Danupon Nanongkai, Richard Peng, and Thatchaphol Saranurak. A deterministic algorithm for balanced cut with applications to dynamic connectivity, flows, and beyond. In Sandy Irani, editor, *61st IEEE Annual Symposium on Foundations of Computer Science, FOCS 2020, Durham, NC, USA, November 16-19, 2020*, pages 1158–1167. IEEE, 2020.

[6] Camil Demetrescu and Giuseppe F. Italiano. A new approach to dynamic all pairs shortest paths. *J. ACM*, 51(6):968–992, 2004.

[7] Camil Demetrescu and Giuseppe F. Italiano. Mantaining dynamic matrices for fully dynamic transitive closure. *Algorithmica*, 51(4):387–427, 2008.

[8] D. Eppstein. Offline algorithms for dynamic minimum spanning tree problems. *Journal of Algorithms*, 17(2):237–250, 1994.

[9] Shimon Even and Yossi Shiloach. An on-line edge-deletion problem. *Journal of the ACM*, 28(1):1–4, 1981.

[10] Greg N. Frederickson. Data structures for on-line updating of minimum spanning trees, with applications. *SIAM J. Comput.*, 14(4):781–798, 1985.

[11] Kathrin Hanauer, Monika Henzinger, and Christian Schulz. Fully dynamic single-source reachability in practice: An experimental study. In *2020 Proceedings of the Twenty-Second Workshop on Algorithm Engineering and Experiments (ALENEX)*, pages 106–119. SIAM, 2020.

[12] Monika Henzinger, Sebastian Krinninger, and Danupon Nanongkai. Sublinear-time decremental algorithms for single-source reachability and shortest paths on directed graphs. In David B. Shmoys, editor, *Symposium on Theory of Computing, STOC 2014, New York, NY, USA, May 31 - June 03, 2014*, pages 674–683. ACM, 2014.

[13] Monika Henzinger, Sebastian Krinninger, Danupon Nanongkai, and Thatchaphol Saranurak. Unifying and strengthening hardness for dynamic problems via the online matrix-vector multiplication conjecture. In Rocco A. Servedio and Ronitt Rubinfeld, editors, *Proceedings of the Forty-Seventh Annual ACM on Symposium on Theory of Computing, STOC 2015, Portland, OR, USA, June 14-17, 2015*, pages 21–30. ACM, 2015.

[14] Monika Henzinger, Barna Saha, Martin P Seybold, and Christopher Ye. On the complexity of algorithms with predictions for dynamic graph problems. In *15th Innovations in Theoretical Computer Science Conference (ITCS 2024)*, pages 62–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2024.

[15] Monika Rauch Henzinger and Valerie King. Randomized fully dynamic graph algorithms with polylogarithmic time per operation. *J. ACM*, 46(4):502–516, 1999.

[16] Jacob Holm, Kristian de Lichtenberg, and Mikkel Thorup. Poly-logarithmic deterministic fully-dynamic algorithms for connectivity, minimum spanning tree, 2-edge, and biconnectivity. *J. ACM*, 48(4):723–760, 2001.

[17] Giuseppe F. Italiano. Amortized efficiency of a path retrieval data structure. *Theoretical Computer Science*, 48:273–281, 1986.

[18] Bruce M. Kapron, Valerie King, and Ben Mountjoy. Dynamic graph connectivity in polylogarithmic worst case time. In *SODA*, pages 1131–1141, 2013.

[19] Adam Karczmarz and Jakub Łącki. Fast and simple connectivity in graph timelines. In Frank Dehne, Jörg-Rüdiger Sack, and Ulrike Stege, editors, *Algorithms and Data Structures*, pages 458–469, Cham, 2015. Springer International Publishing.

[20] Valerie King. Fully dynamic algorithms for maintaining all-pairs shortest paths and transitive closure in digraphs. In *Proceedings of the 40th Annual Symposium on Foundations of Computer Science*, FOCS '99, pages 81–, 1999.

[21] Jakub Lacki. Improved deterministic algorithms for decremental reachability and strongly connected components. *ACM Trans. Algorithms*, 9(3):27:1–27:15, 2013.

[22] Quanquan C Liu and Vaidehi Srinivas. The predicted-updates dynamic model: Offline, incremental, and decremental to fully dynamic transformations. In *The Thirty Seventh Annual Conference on Learning Theory*, pages 3582–3641. PMLR, 2024.

[23] Samuel McCauley, Benjamin Moseley, Aidin Niaparast, and Shikha Singh. Incremental topological ordering and cycle detection with predictions. In *Proceedings of the 41st International Conference on Machine Learning*, pages 35240–35254, 2024.

[24] Michael Mitzenmacher and Sergei Vassilvitskii. *Algorithms with Predictions*, page 646–662. Cambridge University Press, 2021.

[25] Michael Mitzenmacher and Sergei Vassilvitskii. Algorithms with predictions. *Commun. ACM*, 65(7):33–35, June 2022.

[26] Richard Peng, Bryce Sandlund, and Daniel D. Sleator. Optimal offline dynamic 2, 3-edge/vertex connectivity. In Zachary Friggstad, Jörg-Rüdiger Sack, and Mohammad R Salavatipour, editors, *Algorithms and Data Structures*, pages 553–565, Cham, 2019. Springer International Publishing.

[27] Liam Roditty. Decremental maintenance of strongly connected components. In *Proceedings of the Twenty-Fourth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2013, New Orleans, Louisiana, USA, January 6-8, 2013*, pages 1143–1150, 2013.

[28] Liam Roditty and Uri Zwick. On dynamic shortest paths problems. *Algorithmica*, 61(2):389–401, 2011.

[29] Mikkel Thorup. Worst-case update times for fully-dynamic all-pairs shortest paths. In *Proceedings of the 37th Annual ACM Symposium on Theory of Computing, STOC 2005*, pages 112–119, 2005.